

최하림 Harim Choi

머신러닝 엔지니어 | 예측 · 불확실성 · 프로덕션 ML

서울 강북구 | 2.harim.choi@gmail.com

github.com/HarimxChoi | harimxchoi.github.io | linkedin.com/in/harimxchoi

한국어 (native), 영어 (professional working proficiency)

요약

현실을 벡터화하고 예측하며, 불확실성을 추정하는 것에 관심이 있는 머신러닝 엔지니어입니다. 7 년간 공공조달·건설·리테일 데이터를 다루며 Tabular ML, NLP, Computer Vision 과 LLM Agent 를 데이터 수집부터 평가·배포까지 연결해 왔습니다. 낯선 도메인과 기존 시스템의 구조를 빠르게 파악하고, 데이터·모델·운영 흐름을 실제 서비스로 구현해 왔습니다. 정확도뿐 아니라 결과의 신뢰도·비용·운영 제약을 함께 만족하는 시스템을 만듭니다.

핵심 역량

- 빠른 문제 이해와 실행 | 낯선 도메인과 기존 시스템의 구조를 빠르게 파악하고, 중요한 문제부터 우선순위를 정해 운영 가능한 ML 시스템으로 구현합니다.
- 모델 한계 진단과 개선 | 기존 구현의 오류와 성능 저하 원인을 실험과 분석으로 확인하고, 운영 환경과 업무 요구에 맞게 학습·추론·평가 방식을 개선합니다.
- 불확실성을 반영한 의사결정 | 하나의 예측값만 제공하지 않고 분포·신뢰구간·calibration-simulation 을 이용해 결과의 신뢰도와 위험을 함께 제공합니다.
- 연구 방법의 실무 적용 | 새로운 연구 방법을 실제 데이터와 서버·비용·운영 조건에 맞게 조정하고, 반복 실행 가능한 pipeline 으로 구현합니다.
- End-to-End ML 구현 | 데이터 구축·학습·평가·batch-API-서비스 운영을 연결하고, 다른 구성원이 검토하고 이어서 운영할 수 있는 형태로 정리합니다.

핵심 기술

- Tabular & Probabilistic ML | TabPFN · TabICLv2 · Prior-Data Fitted Networks · Tabular ICL · AutoGluon · Quantile/Distributional Prediction · Conformal Prediction · Calibration · Monte Carlo Simulation
- NLP & Document Intelligence | Transformers · PEFT/LoRA · Weak Supervision · Knowledge Distillation · OCR · Document Parsing · Information Extraction
- Computer Vision | Recognition · Object Detection & Tracking · Segmentation · 2D/3D Pose · Motion & Trajectory Prediction
- RAG & Agentic Systems | Vector Retrieval · Hybrid Search · Reranking · LangGraph · MCP · Tool Calling · HITL
- Efficient Model Inference | INT3/INT8 Quantization · Weight/KV/Activation Compression · ONNX CPU/GPU Inference
- Production ML Systems | Data/Training Pipelines · Parallel Inference · API/Batch Serving · Model/Artifact Lifecycle · Monitoring

세종분석연구원 (Sejong Bid Institute) | 연구원

2026.04-현재

공공조달 확률예측·Document AI 시스템 재구축 및 운영

- **확률예측·MLOps** | 기존 5인 프로젝트팀이 6개월간 개발한 공공조달 AI 시스템을 인수해 3개월 안에 재구축했습니다. 지역별 경쟁사 투찰분포를 분위모델+CQR/R2CCP로 모델링하고, TabiCLv2-CatBoost·XGBoost·AutoGluon q1000 model zoo에서 광고당 약 8만 개 후보를 생성했습니다. Monte Carlo simulation으로 후보별 위험·유효확률·낙찰확률을 계산하고 Airflow DAG, 병렬 GPU 추론, artifact checksum, dry-run과 승인 후 배포까지 운영했습니다. 매일 1,000개 이상 고객사에 향후 2일 내 입찰 가능한 광고의 추천가격과 낙찰확률을 제공하며, 기존 단일 분위모델 대비 내부 상대 낙찰 KPI를 30% 개선하고 추론시간을 136초에서 25초로 줄였습니다.
- **Document AI OCR** | HWP/HWPX·PDF 광고문을 수집해 rhwp·hwp5·liteparse v2로 파싱하고 JSON으로 정규화했습니다. Gemini 2.5 Flash-Lite는 주요 필드 복사에만 사용하고 금액·비율·단위는 코드로 검증하는 결정론적 경로를 구성했습니다. 매일 약 2,000개 신규 광고를 처리하며 주요 필드 추출 F1 0.985를 달성했습니다.

한맥그룹 | ML 엔지니어 / 연구원

2024.11-2026.02

공공조달 NLP·다봉형 확률예측·RAG 수량산출 Agent 개발

- **다봉형 PQ 낙찰예측** | PQ 경쟁환경을 8개 context로 분리하고 R2CCP로 다봉형 투찰분포를 모델링했습니다. 공개 구현체의 interval collapse를 entropy regularization과 per-bin conformal threshold로 수정해 떨어진 유효구간을 보존하고, 50만 회 Monte Carlo simulation으로 투찰범위별 낙찰확률을 계산했습니다. 69,934건으로 학습한 8개 모델이 시간순 validation 13,984건에서 coverage 90.73%를 기록했고 그룹사 PQ 낙찰 KPI를 35% 개선했습니다.
- **공공조달 NLP API** | RoBERTa-large+LoRA로 입찰가능성 binary model을 만들고, UMAP·HDBSCAN ontology와 SBERT·RoBERTa teacher 기반 weak labeling으로 세부분야 데이터를 생성했습니다. Binary-multiclass model을 static INT8 ONNX 약 330MB로 경량화해 FastAPI batch path로 CPU 배포했습니다. 입찰가능성 Macro-F1 0.9639·Accuracy 96.4%, 세부분야 F1 0.90과 광고당 약 50ms 추론을 기록했습니다.
- **원가계산 RAG Agent** | 정부 문서 OCR 결과와 표준품셈·단가 데이터를 FAISS 기반 Vector DB/index로 구성했습니다. LangChain retrieval과 LangGraph workflow에서 검색 결과를 원가 계산, 수량산출서와 공문 초안 생성까지 연결했습니다.

산하종합기술 | AI / 자동화 엔지니어

2021.01-2024.11

건설 문서, CAD, ERP와 측량 workflow의 단독 자동화

- **문서 자동화** | 형식이 다른 건설 문서를 OCR·layout parsing으로 구조화하고 confidence 기반 검토와 규칙 기반 후처리를 적용했습니다.
- **CAD·ERP 자동화** | 반복 도면 작업을 줄이는 AutoCAD add-in과 template automation을 개발하고 문서·도면 결과를 ERP 입력 흐름에 연결했습니다.
- **드론·측량 3D workflow** | 드론 이미지와 측량 데이터를 현장 검토용 3D 결과로 변환하는 workflow를 구축했습니다.
- **성과** | 4시간 이상 걸리던 문서 작업을 약 30분대로 단축하고 외주·수작업 문서와 도면 업무를 사내 자동화 도구로 전환했습니다.

닐슨코리아 GTC | 데이터 분석가

2019.08-2020.10

리테일 상품 이미지·바코드 데이터 처리 자동화

- **Retail Vision** | 글로벌 리테일 상품 이미지에서 업무에 필요한 시각 속성을 추출하고 품질을 검사했습니다.
- **Barcode Data** | 바코드 인식 결과를 상품 데이터베이스와 연결해 이미지·상품정보 처리 흐름을 구축했습니다.
- **RPA & Exception Routing** | 반복 입력과 검수 단계를 자동화하고 예외 항목만 별도 검토하도록 분리해 부서 전체 처리시간을 약 40% 단축했습니다.

프로젝트 및 연구

실무 프로젝트

Uncertainty-Aware Bid ML

요약 고객사별 낙찰확률과 불확실성을 함께 제공하는 q1000-Monte Carlo 의사결정 ML

기술 Python · Airflow · DAG · TabICLv2 · CatBoost · XGBoost · AutoGluon · CQR · R2CCP · Monte Carlo Simulation · GPU Inference

- **Why · How** | 추천값 하나만으로는 고객사에게 낙찰 가능성과 위험을 함께 설명할 수 없었습니다. 같은 예측값도 지역·공고별 경쟁환경에 따라 의미가 달랐고, 소수의 분석 인력이 1,000 개 이상의 고객사를 일관된 기준으로 검토하기도 어려웠습니다. 하나의 숫자를 반환하는 모델이 아니라 추천값, 낙찰확률과 불확실성을 함께 제공하는 의사결정 시스템이 필요했습니다. 지역별 경쟁사의 투찰 패턴을 XGBoost+CQR 과 R2CCP 분포로 모델링했습니다. TabICLv2, CatBoost, XGBoost 와 AutoGluon 으로 q1000 분위예측 model zoo 를 구성하고, 공고마다 약 8 만 개의 후보 투찰값을 생성했습니다. 후보들을 지역별 Monte Carlo simulation 으로 평가해 유효확률, 최저가 확률과 예상 낙찰확률을 계산하고 고객사별 추천값을 만들었습니다. 학습, GPU 추론, 리포트와 검증은 자동화된 pipeline 으로 운영했습니다.
- **Result** | 기존 단일 분위모델 대비 낙찰 KPI 를 30% 개선했습니다. 시뮬레이션과 평가 연산을 벡터화해 추론 시간을 136 초에서 25 초로 줄였으며, 고객사별 위험과 낙찰확률을 함께 제공하는 분석 리포트를 구축했습니다.

Document AI OCR

요약 HWP·PDF 나라장터 공고문 자동 추출 · F1 0.985 · 일 2,000 건

기술 Python · HWP/HWPX · PDF · ZIP · rhwp · hwp5 · liteparse v2 · Gemini 2.5 Flash-Lite · JSON · ThreadPool

- **Why · How** | 모델 학습과 공동도급 매칭에 필요한 분야별 지분과 시공능력 기준은 공고 API 가 아니라 첨부된 HWP·PDF 안에만 있었습니다. 하루 약 2,000 개의 신규 문서를 사람이 직접 열어보는 방식으로는 feature 생성과 고객사 매칭을 실시간으로 처리할 수 없었습니다. 문서를 자동으로 구조화하면서도 금액과 비율은 다시 계산하고 확인할 수 있는 처리 경로가 필요했습니다. 신규 HWP/HWPX 와 PDF 를 수집하고 rhwp, hwp5, liteparse v2 로 본문을 추출해 JSON 으로 정규화했습니다. 경량 LLM 에는 필요한 필드의 복사만 맡기고, 금액·비율·단위 변환은 코드로 계산하고 검증했습니다. 이를 통해 같은 입력에는 같은 결과를 반환하는 Document AI pipeline 을 구축했습니다.
- **Result** | 매일 약 2,000 개 공고를 자동 처리하고, 공고문 주요 필드 추출에서 F1 0.985 를 달성했습니다.

Procurement NLP

요약 공고명만으로 입찰가능성과 세부분야를 분류하는 CPU NLP API

기술 Python · PyTorch · KLUE RoBERTa-large · LoRA/PEFT · SBERT · UMAP · HDBSCAN · ONNX Runtime INT8 · FastAPI

github.com/HarimxChoi/nlp-analysis-agent

- **Why · How** | 실시간으로 들어오는 공고를 담당 부서에 곧바로 연결하려면 공고명만으로 입찰 가능 여부와 세부 분야를 판단해야 했습니다. 하지만 하천·소방·정보통신 같은 세부분야 라벨은 거의 없었고 운영 환경에는 추론용 GPU 도 없었습니다. 모든 공고를 수작업으로 라벨링하지 않고 도메인 분류체계를 만들며, 두 가지 판단을 CPU 에서 빠르게 제공하는 pipeline 이 필요했습니다. 먼저 RoBERTa-large+LoRA binary model 로 입찰 가능 여부를 학습했습니다. 그룹사 부서별 과거 공고명을 UMAP·HDBSCAN 으로 군집화해 분야별 ontology 를 만들고, binary teacher model 의 도메인 표현과 SBERT-RoBERTa weak labeling 을 이용해 multiclass model 을 학습했습니다. 두 모델은 INT8 ONNX 로 양자화해 CPU 서버의 FastAPI batch API 로 배포했습니다.
- **Result** | 입찰가능성 모델은 Macro-F1 0.9639, 세부분야 분류모델은 F1 0.90 을 기록했습니다. 두 모델을 CPU 에서 공고당 약 50ms 로 추론했습니다.

R2CCP Bid Prediction

요약 8 개 경쟁환경의 다봉형 PQ 투찰분포 모델링

기술 Python · PyTorch · R2CCP · Per-bin Conformal Threshold · Entropy Regularization · Monte Carlo Simulation

github.com/HarimxChoi/ensemble-bid-prediction

- **Why · How** | PQ 입찰은 기술점수와 가격점수를 함께 평가합니다. 기술점수가 낮아질수록 유효한 가격대가 입찰 불가능 구간을 사이에 두고 갈라져 실제 투찰행태가 하나의 봉우리가 아닌 다봉분포로 나타났습니다. 하나의 예측값이나 합쳐진 구간이 아니라 서로 떨어진 유효구간을 보존하고 후보별 낙찰확률을 계산하는 시스템이 필요했습니다. 경쟁사의 가격점수와 투찰행태를 특징으로 사용해 경쟁환경을 8 개 context 로 분리하고, context 별 R2CCP 분포를 학습했습니다. 기존 구현이 두 봉우리와 그 사이의 빈 구간을 하나의 interval 로 합치는 문제를 발견해 entropy regularization 과 per-bin conformal threshold 를 적용했습니다. 이후 후보별 50 만 회 Monte Carlo simulation 으로 투찰 범위별 예상 낙찰확률을 계산했습니다.
- **Result** | 69,934 건으로 8 개 context model 을 학습하고, 13,984 건의 시간순 validation 에서 coverage 90.73%를 기록했습니다. 실제 운영에서는 전체 그룹사의 PQ 낙찰 KPI 를 35% 개선했습니다.

모델 효율화-Agent 시스템

WarpQuant

요약 Weight-KV Cache-Activation 을 함께 압축하는 INT3 LLM Quantization

기술 Python · PyTorch · Hadamard Rotation · Block-GPTQ · Output-Fisher · TurboQuant KV Cache · GGUF

github.com/HarimxChoi/WarpQuant

- **Why · How** | TurboQuant 가 Hadamard rotation 으로 KV Cache 의 outlier 를 완화하는 방식이 정말 놀랍다고 생각했습니다.. 해당 이론을 weight quantization 에 적용하면 낮은 bit 에서도 정확도 손실을 줄일 수 있다고 보고, Weight-KV Cache-Activation 을 함께 압축하는 방법을 연구했습니다. Hadamard rotation 으로 outlier 를 분산하고 block-GPTQ 로 INT3 weight 를 생성했습니다. Output-Fisher 로 next-token loss 에 민감한 column 만 선택해 복원하고, KV Cache 에는 TurboQuant 를 적용했으며 token-wise activation quantization 을 별도 경로로 구성했습니다.
- **Result** | Qwen3.8-27B text backbone 을 3.6165 bpw-11.32 GiB 로 압축했습니다. MMLU 와 상식 평가에서는 BF16 성능의 약 99%를 유지했고, ARC 는 BF16 52.1%보다 +4.7%, 56.8%로 높아졌습니다. PPL 은 원본 대비 7.5% 악화되었습니다.

google-surf-mcp

요약 LLM Agent 를 위한 검색·웹·학술 PDF 통합 MCP

기술 TypeScript · MCP · Google Search · Google Scholar · Playwright · Readability · Spatial PDF Parsing · SSRF Protection · Vitest

github.com/HarimxChoi/google-surf-mcp

- **Why · How** | LLM Agent 가 유용한 답을 내려면 검색 결과 링크뿐 아니라 최신 웹페이지와 학술 PDF 의 본문까지 읽을 수 있어야 합니다. 기존 tool 은 일반검색·학술검색·본문 추출이 분리돼 느렸고, 검색 품질과 PDF parsing 도 불안정했습니다. Agent 가 하나의 interface 에서 근거를 검색하고 읽으며 실패까지 복구할 수 있는 검색 인프라가 필요했습니다. Google Search 와 Google Scholar 를 하나의 MCP 에서 사용할 수 있게 하고, 여러 검색어를 동시에 실행하는 병렬검색을 구현했습니다. 검색 결과의 웹페이지는 Playwright 와 Readability 로 본문을 추출하고, 학술 PDF 는 문서의 좌표와 읽기 순서를 복원해 parsing 했습니다. 검색 provider 가 실패하면 다른 경로로 전환하고, CAPTCHA 가 나타나면 사용자가 직접 해결한 뒤 작업을 이어갈 수 있도록 만들었습니다.
- **Result** | Agent 가 하나의 MCP 에서 일반검색, 학술검색, 웹페이지 추출과 PDF parsing 까지 처리할 수 있게 됐습니다. 검색부터 본문 확보까지 걸리는 tool 호출 횟수를 줄였고, 검색·추출·복구·보안 기능을 다루는 388 개 test 를 통과했습니다. npm 패키지로 공개했으며 MCP TOPLIST Top 1%에 진입했습니다.

Monogram

요약 뉴스·SNS·논문·영상을 공유 한 번으로 정리하고 검색하는 개인 지식관리 시스템

기술 Python · 5-stage LLM Pipeline · Atomic Git Tree · Markdown · Telegram · GitHub API · MCP · Dashboard

github.com/HarimxChoi/monogram

- **Why · How** | 뉴스, SNS, arXiv, YouTube 와 학술 리포트에서 본 정보가 여러 앱과 북마크에 흩어져 필요할 때 다시 찾기 어려웠습니다. 어떤 형태의 지식이든 공유 버튼 한 번으로 수집하고, 왜 중요했는지와 출처를 보존하며 자동으로 정리·분류·검색하고 싶었습니다. 사람과 Agent 가 같은 지식을 다시 사용할 수 있는 개인 지식관리 시스템이 목표였습니다. 사용자가 공유한 링크, 메시지, 문서와 코드를 하나의 입력 경로로 수집했습니다. LLM pipeline 이 원문과 metadata 를 읽고 핵심 내용, 주제, 출처와 관련 지식을 구조화된 Markdown 으로 정리합니다. 정리된 지식은 Git 에 저장해 원문과 변경 이력을 보존하고, Dashboard 와 MCP 를 통해 사용자와 Agent 가 동일한 지식을 검색하고 다시 활용할 수 있게 했습니다.
- **Result** | 뉴스, SNS, 논문, 영상과 문서를 별도로 정리하지 않아도 공유 한 번으로 개인 지식베이스에 저장되는 workflow 를 구현했습니다. 저장된 지식은 출처와 변경 이력이 함께 남으며 Dashboard 에서 검색할 수 있고, 13 개 MCP tool 을 통해 다른 Agent 의 context 로도 사용할 수 있습니다.

Bau Browser

요약 프라이버시와 Agent 의 실행권한을 사용자가 직접 통제하는 Browser

기술 TypeScript · Electron · MCP · SQLite · Qwen3.5-4B · XGrammar · TaskSpec · ActionScope · DecisionLog

- **Why · How** | Chrome 이 무겁고 불편하다고 느꼈고, Agent 를 브라우저에 붙이는 순간 사용자의 프라이버시와 행동 통제권이 더 중요해진다고 생각했습니다. 기존 방식에서는 browsing data 가 어디로 가는지, Agent 가 어떤 사이트에서 무엇을 할 수 있는지 확인하기 어려웠습니다. 개인 데이터를 local 에 두고 Agent 의 권한·승인·실행기록을 사용자가 직접 관리하는 desktop browser 를 만들고 싶었습니다. Electron 기반의 local-first browser 를 만들고 browsing data, Agent memory 와 실행기록을 사용자의 로컬 환경에서 관리하도록 구성했습니다. Agent 는 브라우저를 바로 조작하지 않고 먼저 수행할 작업을 제안합니다. 사용자가 허용한 사이트와 행동 범위 안에서만 실행 인수를 만들고, 클릭·입력·전송처럼 상태를 바꾸는 행동은 승인 후 실행하며 모든 결과를 receipt 와 DecisionLog 로 남겼습니다.
- **Result** | 프라이버시 관리, Agent 행동 승인과 실행기록이 브라우저 자체에 내장된 agent-native desktop MVP 를 구현했습니다. 사용자가 허용한 사이트와 행동만 실행하고, 모든 작업이 제안 → 승인 → 실행 → 결과 확인 과정으로 남도록 설계했습니다. 새로운 도메인으로 구성한 36 개 task 에서 실행할 행동과 인수를 36/36 정확히 생성했으며, 허용 범위를 벗어난 행동은 0 건을 기록했습니다.

LangGraph Travel Agent

요약 자연어 여행요청에서 예약 가능한 3 개 여행상품을 만드는 LangGraph Agent

기술 Python · LangGraph · FastAPI · AsyncIO · InMemorySaver · Amadeus API · Hotelbeds API · HubSpot · Twilio

github.com/HarimxChoi/langgraph-travel-agent

- **Why · How** | 미국 여행사는 고객의 자연어 요청을 받아 예산과 일정에 맞는 항공편·숙소·관광지·맛집을 찾아주는 Agent 를 요청했습니다. 기존 상담에서는 누락 정보 확인, 공급사 검색, 패키지 비교, CRM 저장과 후속 연락을 서로 다른 도구에서 반복해야 했습니다. 첫 요청부터 상담자 검토와 고객 연락까지 같은 맥락을 유지하는 workflow 가 필요했습니다. Agent 가 고객의 출발지, 목적지, 일정, 인원, 예산과 선호도를 먼저 구조화하도록 만들었습니다. 항공편, 호텔, 관광지와 음식점 정보를 여러 API 에서 동시에 수집하고, 이동시간·가격·고객 조건을 비교해 Budget, Balanced, Premium 세 가지 패키지를 생성했습니다. 생성된 결과는 상담자가 검토할 수 있도록 저장하고 HubSpot CRM 과 Twilio 를 연결해 고객정보, 상담상태와 후속 연락이 같은 workflow 에서 이어지게 했습니다.
- **Result** | 자연어 요청 접수부터 여행상품 생성, 상담자 검토, CRM 저장과 고객 연락까지 이어지는 여행사 workflow 를 구현했습니다. 미국 여행사에 납품했으며, 이후 계약에 따라 프로젝트 IP 를 이전받아 공개 OSS 로 정리했습니다.

Weakly Supervised Semantic Segmentation

요약 WSSS SOTA 연구용역 · COCO-Val mIoU 53.31% · WeCLIP+ 대비 +1.5%p

기술 Python · PyTorch · CLIP ViT-B/16 · DINOv2 · WeCLIP+ · RFM · COCO-Val

github.com/HarimxChoi/wsss-refined-pseudolabels

- **Why · How** | 클라이언트로부터 이미지 단위 정답만 사용해 당시 SOTA 였던 WeCLIP+의 성능을 개선하는 연구용역을 의뢰받았습니다. 자동으로 만든 pseudo-mask에는 객체 경계와 배경의 잘못된 pixel이 포함됐고, 이 오류는 self-training을 반복할수록 누적됐습니다. 전체 mask를 다시 만드는 대신 신뢰하기 어려운 pixel만 찾아 복원할 수 있는지가 핵심 연구 질문이었습니다. CLIP과 DINOv2 backbone은 고정하고, 두 모델의 표현을 결합하는 fusion head와 decoder만 학습했습니다. 두 표현이 서로 다르게 판단한 pixel을 불신된 영역으로 정의하고 이를 복원한 뒤, 수정한 pseudo-mask를 self-training에 다시 사용했습니다.
- **Result** | COCO-Val 2014 전체 40,137개 이미지에서 mIoU 53.31%를 기록했습니다. 당시 SOTA 모델이었던 WeCLIP+ ViT-B/16의 51.8%보다 1.5%p 높은 결과를 달성했습니다.

Vargo

요약 실행 전에 어떤 Agent 설정이 가장 좋은지 예측할 수 있는가?

기술 Python · ALFWorld · 134×29 Evaluation Matrix · Router · Bandit · Activation Probe · Verifier · Sealed-cell Evaluation

- **Why · How** | Agent의 scaffold, memory, retry와 verifier 조합은 task마다 최적값이 달랐습니다. 그렇다면 task text, embedding이나 hidden state만 보고 실행 전에 가장 좋은 구성을 선택할 수 있을까? 그것이 어렵다면 제한된 후보를 직접 실행하고 검증하는 방식이 최적 성능과의 차이를 얼마나 회복할 수 있는지가 핵심 연구 질문이었습니다. 같은 134개 task를 29개의 서로 다른 Agent 설정으로 각각 실행해 task마다 실제로 가장 잘 작동한 설정을 기록했습니다. 이후 task text, embedding, hidden state와 과거 실행 결과를 사용하는 여러 선택 방법이 실제 최적 설정을 얼마나 잘 찾아내는지 비교했습니다. 마지막으로 실행 전에 하나를 고르는 방식과, 여러 후보를 실제로 실행한 뒤 verifier가 성공 여부를 확인하는 방식을 비교했습니다.
- **Result** | Task text, embedding과 hidden state만으로는 실행 전에 최적 Agent 설정을 안정적으로 선택할 수 없었습니다. 반면 가능성 높은 설정을 직접 실행하고 결과를 검증하자 최적 성능과의 격차 회수율이 54%에서 85%로 높아졌습니다. 실제 배포 가능한 11개 후보 설정에서는 최적 성능과의 차이를 100% 회수했습니다. 이 결과를 바탕으로 논문을 작성해 TMLR에 제출했으며 현재 review 중입니다.

EMH Agent

요약 강화학습 투자 Agent(TQC)

기술 Python · TQC · IVV/IEF/SHV · Point-in-time Panel · Information Agent · Transaction Costs · Drawdown

github.com/HarimxChoi/emh-agent

- **Why · How** | AI가 사람의 감정에 흔들리지 않고 더 일관된 투자 판단을 내릴 수 있을지 궁금했습니다. 이 질문을 제대로 확인하려면 미래 정보를 보지 않고, 각 시점까지 누적된 정보만으로 판단하며 거래비용과 slippage까지 반영해야 했습니다. 결과 숫자만 좋은 backtest가 아니라 같은 조건에서 다시 실행할 수 있는 투자 의사결정 과정을 만들고자 했습니다. 시장정보 수집 Agent가 가격, 위험과 관련 정보를 지속해서 수집하고, 의사결정 모델은 각 시점까지 확인할 수 있었던 정보만 사용하도록 분리했습니다. TQC 기반 RL model이 IVV, IEF와 SHV의 투자비중을 결정하도록 학습하고, 세 개의 seed 결과를 결합했습니다. 평가에서는 거래비용, slippage와 차입비용을 차감하고 같은 기간의 S&P 500 ETF와 수익률-drawdown을 비교했습니다.
- **Result** | 2019년부터 2021년까지 756거래일의 비용 반영 backtest에서 TQC Agent는 누적수익률 +183.9%, S&P 500 ETF인 IVV는 +99.6%를 기록했습니다. 같은 기간 maximum drawdown은 TQC -14.2%, IVV -33.8%였습니다.

MyShot

요약 스마트폰 영상에서 스윙 자세를 공의 속도·방향·비거리까지 연결하는 3D Golf Vision

기술 Python · PyTorch · RTMPose · MotionAGFormer · YOLO11 · GolfPose · GVHMR · OpenCV · Vicon · Qwen2.5-VL

- **Why · How |** 골프 레슨과 장비는 비용이 높고, 혼자 연습할 때는 자세가 왜 잘못됐는지뿐 아니라 그 움직임이 공의 속도·방향·비 거리에 어떤 영향을 주는지 알기 어렵습니다. 일반 pose model 은 몸의 움직임을 보여주는 데서 끝나기 때문에, 스마트폰 영상 하나로 스윙 자세와 실제 샷 결과를 함께 분석하는 시스템을 만들고 싶었습니다. 영상에서 골퍼·클럽·공을 검출·추적하고, RTMPose 의 2D 관절을 MotionAGFormer 기반 모델로 변환해 전체 스윙의 3D 움직임을 복원했습니다. 복원한 자세에서 X-Factor, 무릎 움직임, swing power 와 clubhead speed 를 계산하고, 사용자 체격과 클럽별 carry-ball-speed 기록을 개인 보정값으로 연결하도록 설계했습니다. 이후 physics-based ball-flight model 로 ball speed, 시작 방향과 carry 를 계산하며, pose disagreement-jitter-reprojection error 로 신뢰하기 어려운 프레임을 구분했습니다.
- **Result |** GolfPose Vicon clean-2D evaluation 에서 3D 관절오차 MPJPE 35.6mm, X-Factor MAE 3.1°와 clubhead-speed 중앙오차 6.1%를 기록했습니다. AIHub 영상 50 개에서 25 개의 carry GT pair 를 구축하고 physics-based ball-flight pipeline 까지 연결했습니다. 다만 30fps body motion 만으로 carry 를 직접 회귀하는 신호는 약해, ball-club tracking 과 클럽별 개인 calibration 을 결합하는 구조로 개선했습니다.

E-AT

요약 ISIC 피부병변 분류 · Minimum ECE 1.03% vs 당시 SOTA CR-SAM (AAAI 2024) 1.7%

기술 Python · PyTorch · ConvNeXtV2-Tiny · Focal Loss · R-Drop · FGM · ISIC 2019 · Reliability Diagram

- **Why · How |** 의료 이미지 모델은 정답을 맞히는 것만큼 틀릴 때 얼마나 확신하는지도 중요합니다. 정확도가 비슷해도 틀린 예측에 높은 confidence 를 주는 모델은 사람이 결과를 과신하게 만들 수 있습니다. 분류 성능을 유지하면서 예측 confidence 의 신뢰도까지 같은 학습 과정에서 개선할 수 있는지를 연구했습니다. ISIC 2019 4-class balanced dataset 에서 ConvNeXtV2-Tiny 를 사용했습니다. 어려운 sample 에 가중치를 높이는 Focal Loss, 두 stochastic forward 의 예측을 일치시키는 R-Drop 과 parameter perturbation 을 적용하는 FGM 을 결합해 E-AT 학습을 구성했습니다. Epoch 별 macro-F1 과 15-bin ECE 를 함께 추적했습니다.
- **Result |** Validation 에서 최고 macro-F1 0.8647 을 기록했고, minimum-ECE checkpoint 에서는 ECE 1.03%로 당시 SOTA method 인 CR-SAM (AAAI 2024)의 1.7%보다 0.67%p 낮췄습니다. 분류 성능과 confidence calibration 을 하나의 학습 framework 에서 함께 평가했습니다.